Handling Missing Values Techniques in Python

Theory + Complete Single Python Program

Missing Values

- • Missing values are represented as NaN.
- • They can reduce model accuracy.
- • Common techniques: Detect, Remove, Replace, Forward Fill, Backward Fill, Interpolation.

Missing Values

Python Code:

```
import pandas as pd

# Dataset with missing values
data = {
    'Name': ['Asif', 'Rahul', None],
    'Age': [20, None, 22]
}

df = pd.DataFrame(data)

print("Missing Values:")
print(df.isnull().sum())
```

Output:

```
Missing Values:
Name    1
Age     1
dtype: int64
```

Summary

- Detect : isnull(), isnull().sum()
- Remove : dropna()
- Replace : fillna()
- Estimate : interpolate()
- Forward Fill : ffill()
- Backward Fill : bfill()

Imputation Techniques

- Imputation is the process of replacing missing values with estimated values instead of deleting data.

- Common Imputation Techniques:
- • Mean Imputation: df['Age']=df['Age'].fillna(df['Age'].mean())
- • Median Imputation: df['Salary']=df['Salary'].fillna(df['Salary'].median())
- • Mode Imputation: df['City']=df['City'].fillna(df['City'].mode()[0])
- • Constant Value: df.fillna(0)
- • Forward Fill: df.ffill()
- • Backward Fill: df.bfill()
- • Interpolation: df.interpolate()

- Advantages:
- • Preserves dataset size
- • Improves model performance
- • Reduces information loss

Imputation Technique - Complete Python Code

- import pandas as pd
- import numpy as np
- from sklearn.impute import SimpleImputer

- data = {
-     'Age':[25,np.nan,30,28,np.nan],
-     'Salary':[50000,60000,np.nan,55000,52000],
-     'City':['Chennai','Coimbatore',np.nan,'Madurai','Chennai']
- }

- df = pd.DataFrame(data)

- # Mean Imputation
- df[['Age']] = SimpleImputer(strategy='mean').fit_transform(df[['Age']])

- # Median Imputation
- df[['Salary']] = SimpleImputer(strategy='median').fit_transform(df[['Salary']])

- # Mode Imputation
- df[['City']] = SimpleImputer(strategy='most_frequent').fit_transform(df[['City']])

- # Constant Value Imputation
- marks = pd.DataFrame({'Marks':[85,np.nan,90,np.nan,75]})
- marks[['Marks']] = SimpleImputer(strategy='constant',
- fill_value=0).fit_transform(marks[['Marks']])

- print(df)
- print(marks)